

DEVELOPER WORKFLOW & CLAUDE PROMPTS

Courier Management System — Build Guide

Module-by-module build plan with ready-to-use Claude Code prompts

Project	Courier Management System (MVP)
Total Modules	9 Modules (3 Frontend + 5 Backend + 1 Deploy)
Strategy	Frontend-first (mock data) → Backend module → Integrate
Total Duration	45 Days
Stack	Next.js 14 (App Router) + Django 5 + PostgreSQL + Celery/Redis

1. Build Strategy Overview

This guide breaks the entire Courier Management System into 9 self-contained modules, each with a clear goal, task list, and a ready-to-paste Claude Code prompt. Follow the modules in order — each one builds on the previous.

1.1 Core Philosophy

- ▶ Build the entire frontend first using mock/static data — no backend dependency
- ▶ Then build backend modules one at a time — starting with Auth, then Merchants, then Parcels
- ▶ After each backend module, replace the matching mock data in the frontend with real API calls
- ▶ This keeps both frontend and backend independently testable at every stage

1.2 How to Use the Claude Prompts

1. Open Claude Code (claude.ai or VS Code extension) in your project root
2. Copy the Claude Prompt from the relevant module section
3. Paste it into Claude Code and let it generate the files
4. Review the output, run the dev server, and verify the feature works
5. Commit to Git before moving to the next module

Note: Each prompt is self-contained and provides full context. You do not need to chain prompts — Claude Code will read your existing files automatically.

1.3 Master Module Timeline

ID	Days	Phase	Module	Key Output
M1	Days 1–5	Frontend	Project Scaffold & Shared UI	Bootstrap the Next.js 14 project with folder structure, global layout, design system token...
M2	Days 6–10	Frontend	Admin Dashboard UI	Build all Admin dashboard pages with full UI using mock data — merchant list, parcel manag...
M3	Days 11–14	Frontend	Merchant Dashboard UI	Build all Merchant dashboard pages — parcel booking form, parcel list, tracking detail, CO...
M4	Days 15–18	Backend	Backend Scaffold + Auth API	Bootstrap Django project with PostgreSQL, JWT auth, role-based user model, and all auth en...
M5	Days 19–22	Backend	Merchant Management API	Build merchant profile model, registration endpoints, and admin management APIs. Integrate...
M6	Days 23–27	Backend	Zone & Parcel API	Build zones, delivery charges, and full parcel CRUD with status management. Integrate with...

M7	Days 28–32	Backend	COD & Payment API + Reports	Build COD disbursement tracking, payment records, and all report endpoints. Integrate with...
M8	Days 33–37	Backend	Email Notifications + PDF Invoice	Build Celery-based email notification system and PDF parcel invoice generation. Integrate ...
M9	Days 38–45	Testing & Deploy	QA, Testing & Production Deployment	Write tests, fix bugs, deploy both backend and frontend to VPS with Nginx, SSL, and system...

1.4 Git Branch Strategy

Use feature branches for each module. Merge to main only after testing integration.

```
main                ← production-ready code only
dev                 ← integration branch
feature/m1-scaffold ← Module 1
feature/m2-admin-ui ← Module 2
feature/m3-merchant-ui ← Module 3
feature/m4-auth-api ← Module 4
...and so on
```

1.5 Environment Setup

Frontend (.env.local)

```
NEXT_PUBLIC_API_URL=http://localhost:8000/api
NEXT_PUBLIC_APP_NAME=Courier CMS
```

Backend (.env)

```
SECRET_KEY=your-secret-key-here
DEBUG=True
DATABASE_URL=postgres://user:password@localhost:5432/courier_cms
REDIS_URL=redis://localhost:6379/0
ALLOWED_HOSTS=localhost,127.0.0.1
CORS_ALLOWED_ORIGINS=http://localhost:3000
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_HOST_USER=your@email.com
EMAIL_HOST_PASSWORD=your-app-password
DEFAULT_FROM_EMAIL=Courier CMS <noreply@cms.com>
ADMIN_EMAIL=admin@cms.com
```

2. Module-by-Module Build Guide

Each module below contains: Goal, What to Build (frontend/backend tasks), and the complete Claude Code prompt to generate the code.

M1 — Project Scaffold & Shared UI

Frontend	Days 1–5	Bootstrap the Next.js 14 project with folder structure, global layout, design system tokens, shared components, and routing skeleton for both Admin and Merchant dashboards.
----------	----------	--

Frontend Tasks

- next.js 14 app router project init with TypeScript + Tailwind CSS
- Folder structure: /app/(admin)/**, /app/(merchant)/**, /app/(auth)/**
- Global layout: sidebar, topbar, breadcrumb, notification bell
- Design tokens: colors, typography, spacing (match proposal palette)
- Shared components: Button, Input, Badge, Table, Modal, Dropdown, Pagination, Skeleton loader, Toast/Alert
- Auth layout: Login page UI (Admin + Merchant — same page, role-based redirect)
- Route guards: ProtectedRoute HOC (checks localStorage token, redirects to /login)
- Axios instance: baseURL from env, request interceptor attaches Bearer token, response interceptor handles 401
- All API calls mocked with static JSON — no real backend yet

Claude Code Prompt

Note: Copy the entire block below and paste it into Claude Code. Run from your project root.

```
You are building a Next.js 14 (App Router, TypeScript) frontend for a Courier Management System. The system has two roles: Admin and Merchant.

TASK: Scaffold the entire project with working UI shell (no real API calls yet — use mock static data).

PROJECT STRUCTURE:
courier-cms-frontend/
├── app/
│   ├── (auth)/login/page.tsx
│   ├── (admin)/
│   │   ├── layout.tsx           ← Admin sidebar + topbar
│   │   ├── dashboard/page.tsx  ← placeholder
│   │   ├── merchants/page.tsx  ← placeholder
│   │   ├── parcels/page.tsx    ← placeholder
│   │   └── ...
│   ├── (merchant)/
│   │   ├── layout.tsx          ← Merchant sidebar + topbar
│   │   ├── dashboard/page.tsx  ← placeholder
│   │   ├── parcels/page.tsx    ← placeholder
│   │   └── ...
│   └── components/ui/          ← Button, Input, Badge, Table, Modal, Pagination, Toast, Skeleton
└── lib/
```

├── axios.ts

├── auth.ts

├── mock-data/

├── types/

├── index.ts

├── hooks/

└── useAuth.ts

← Axios instance with interceptors

← getToken(), setToken(), clearToken() helpers

← static JSON files for each module

← shared TypeScript interfaces

← reads token, returns role + user info

DESIGN:

- Tailwind CSS only (no component library)
- Color palette: primary green #4A6741, dark brown #3B3028, accent amber #C07A3A, bg #F2F5F0
- Sidebar shows nav items filtered by role
- Topbar shows: page title, notification icon, user avatar + name + logout

AUTH FLOW (mock):

- Login page: email + password form
- On submit, check hardcoded credentials:
 - admin@cms.com / admin123 → role: "admin", redirect to /admin/dashboard
 - merchant@cms.com / merchant123 → role: "merchant", redirect to /merchant/dashboard
- Store { token, role, name } in localStorage
- ProtectedRoute checks localStorage on every protected page – redirects to /login if missing

MOCK DATA: Create /lib/mock-data/parcels.json, merchants.json, zones.json with at least 10 realistic Bangladesh-context records each.

OUTPUT: All files with full working code. Sidebar navigation must be functional (active states). Pages can show "Coming soon" placeholder with the correct layout shell.

M2 — Admin Dashboard UI

Frontend	Days 6–10	Build all Admin dashboard pages with full UI using mock data — merchant list, parcel management, zone config, COD management, and reports pages.
----------	-----------	--

Frontend Tasks

- ▶ Admin /dashboard — KPI cards (total parcels, merchants, COD pending, revenue), charts (recharts: daily parcel trend line chart, delivery vs return pie chart)
- ▶ Admin /merchants — data table with search, status filter (pending/active/suspended), approve/reject action buttons, pagination
- ▶ Admin /merchants/[id] — merchant detail page: profile info, parcel history table, COD balance card
- ▶ Admin /parcels — advanced filter bar (status, date range, merchant, zone), bulk select + bulk status update UI
- ▶ Admin /parcels/[id] — parcel detail: timeline stepper, status update dropdown, add remark form
- ▶ Admin /zones — zone list table, create/edit zone modal with charge fields (regular, express, COD%, return charge)
- ▶ Admin /cod — merchant COD balance table, mark-as-paid modal with bank reference + date fields
- ▶ Admin /reports — date range picker, report type selector, export CSV button (downloads mock CSV)
- ▶ Admin /settings — company info form, logo upload UI

© 2025. All rights reserved. Page 5

Claude Code Prompt

Note: Copy the entire block below and paste it into Claude Code. Run from your project root.

```
You are continuing the Courier Management System frontend (Next.js 14, TypeScript, Tailwind CSS).

TASK: Build all Admin dashboard pages with complete UI. Use mock data from /lib/mock-data/. No real API calls yet.

PAGES TO BUILD:

1. /admin/dashboard
  - 4 KPI cards: Total Parcels, Active Merchants, Pending COD (BDT), Delivered Today
  - Line chart (recharts): last 7 days parcel volume
  - Pie chart (recharts): Delivered vs Returned vs Pending
  - Recent parcels table: last 5 with tracking ID, merchant, status badge, date

2. /admin/merchants
  - Search bar + status filter tabs (All / Pending / Active / Suspended)
  - Table: merchant name, shop name, phone, join date, status badge, Actions (View / Approve / Suspend)
  - Approve/Suspend use optimistic UI update on mock data
  - Pagination (10 per page)

3. /admin/merchants/[id]
  - Profile card: avatar initial, name, shop, phone, address, join date, status
  - COD balance card: total collected, disbursed, pending
  - Parcel history table with status badges

4. /admin/parcels
  - Filter bar: status multiselect, date range picker, merchant dropdown, zone dropdown
  - Table with checkbox selection: tracking ID, recipient, merchant, zone, COD amount, status, date
  - Bulk action bar (appears when rows selected): "Update Status" dropdown + Apply button
  - Status badge colors: Pending=amber, Picked Up=blue-ish, Delivered=green, Returned=red, Cancelled=gray

5. /admin/parcels/[id]
  - Parcel info card: tracking ID, recipient details, weight, COD amount, zone
  - Status timeline stepper (vertical): shows history of status changes with timestamp
  - Update status form: select new status + optional remark → updates mock state
  - Internal remarks list

6. /admin/zones
  - Zone cards grid: name, districts covered, regular charge, express charge, COD%, return charge
  - "Add Zone" button → modal form
  - Edit icon on each card → same modal prefilled

7. /admin/cod
  - Table: merchant name, total COD collected, disbursed, pending, last payment date
  - "Mark Paid" button → modal: amount, bank name, reference number, date
  - Paid badge vs Pending badge

8. /admin/reports
  - Report type selector: Parcel Report / Merchant Report / COD Report
  - Date range: start date, end date
  - Generate button → shows table with results from mock data
  - Export CSV button → triggers browser download of CSV string

9. /admin/settings
  - Form: company name, email, phone, address, logo upload preview
  - Save button (mock — shows success toast)
```

COMPONENTS NEEDED:

- StatusBadge: maps status string → correct color
- Timeline: vertical stepper for parcel history
- StatCard: icon + label + value + optional trend arrow
- FilterBar: composable filter row
- ExportButton: generates CSV from array of objects and triggers download

Use recharts for all charts. All modals use a shared Modal component. Keep code clean, typed with TypeScript interfaces.

M3 — Merchant Dashboard UI

Frontend	Days 11–14	Build all Merchant dashboard pages — parcel booking form, parcel list, tracking detail, COD view, and reports.
----------	------------	--

Frontend Tasks

- ▶ Merchant /dashboard — KPI cards (total parcels, delivered, returned, pending COD), recent parcel table
- ▶ Merchant /parcels/new — multi-step booking form: recipient info → parcel details → review & confirm
- ▶ Merchant /parcels — parcel list with status filter, date range, search by tracking ID, export CSV
- ▶ Merchant /parcels/[id] — tracking detail page with timeline stepper, cancel button (if Pending)
- ▶ Merchant /parcels/import — CSV/Excel drag-and-drop upload UI, column mapping preview, confirm import
- ▶ Merchant /cod — COD balance overview card, transaction history table, disbursement status
- ▶ Merchant /reports — date range report with export
- ▶ Merchant /profile — edit profile form: shop name, address, phone, pickup address

Claude Code Prompt

Note: Copy the entire block below and paste it into Claude Code. Run from your project root.

```
You are continuing the Courier Management System frontend (Next.js 14, TypeScript, Tailwind CSS).

TASK: Build all Merchant dashboard pages with complete UI. Use mock data. No real API calls.

PAGES TO BUILD:

1. /merchant/dashboard
  - KPI cards: Total Parcels, Delivered, Returned, Pending COD (BDT)
  - Line chart: my parcel volume last 7 days
  - Recent parcels table: last 5 parcels with status badges + "View" link

2. /merchant/parcels/new ← multi-step form
  Step 1 — Recipient Info: name, phone, full address, district (dropdown of all Bangladesh districts), delivery type (Regular/Express)
  Step 2 — Parcel Details: weight (kg), product description, COD amount (BDT), special instructions
  Step 3 — Review & Confirm: summary card, calculated delivery charge based on zone, COD charge, submit button
  On submit: add to mock parcel list, show success toast with generated tracking ID (e.g. CMS-2025-XXXXX), redirect to /merchant/parcels

3. /merchant/parcels
  - Search by tracking ID
  - Status filter tabs: All / Pending / In Transit / Delivered / Returned / Cancelled
  - Date range filter
  - Table: tracking ID, recipient name, district, delivery type, COD, status badge, created date, Actions (View / Cancel)
  - Cancel: only shows for Pending status, asks confirmation modal
  - Export CSV button

4. /merchant/parcels/[id]
  - Tracking ID + QR code placeholder (just a box with text "QR Code")
  - Recipient details card
  - Parcel details card: weight, COD, delivery type, zone, charges
```

```
- Status timeline stepper
- Cancel button (if status is Pending)
- Download Invoice button (mock - alert "Invoice PDF download coming soon")

5. /merchant/parcels/import
- Drag-and-drop file zone (accepts .csv, .xlsx)
- After file selected: parse CSV headers, show column mapping table (map CSV columns to system fields)
- Preview first 5 rows
- Import button → adds to mock parcel list, shows success count toast

6. /merchant/cod
- Balance summary: Total COD Collected (by courier), Total Disbursed, Current Pending
- Transaction table: date, amount, reference, status badge (Paid/Pending)

7. /merchant/reports
- Date range selector
- Summary: total parcels, delivered, returned, success rate %
- Table with export CSV

8. /merchant/profile
- Form: shop name, owner name, email (read-only), phone, pickup address, district
- Save → success toast

ALSO ADD: Parcel booking charges preview in Step 3 of booking form:
- Delivery charge = mock zone charge (Regular: 60tk Dhaka / 110tk outside)
- COD charge = 1% of COD amount (minimum 10tk)
- Total charge = delivery + COD charge
Show this as a breakdown card.
```

M4 — Backend Scaffold + Auth API

Backend	Days 15–18	Bootstrap Django project with PostgreSQL, JWT auth, role-based user model, and all auth endpoints. Integrate with frontend login.
---------	------------	---

Frontend Tasks

- ▶ Replace mock login with real POST /api/auth/login/ call
- ▶ Store JWT access + refresh tokens in localStorage
- ▶ Axios interceptor: attach Authorization header on all requests
- ▶ Axios response interceptor: on 401, call refresh token endpoint, retry request once, else redirect to /login
- ▶ Update useAuth hook to decode JWT payload for user info (name, role)

Backend Tasks

- ▶ Django 5 project init: courier_cms/, apps: users/, parcels/, zones/, cod/
- ▶ PostgreSQL database config via .env (django-envron)
- ▶ Custom User model: email as USERNAME_FIELD, role field (SUPER_ADMIN, ADMIN, MERCHANT)
- ▶ SimpleJWT config: access token 1 day, refresh token 30 days

- ▶ POST /api/auth/login/ — returns access + refresh + user info
- ▶ POST /api/auth/refresh/ — returns new access token
- ▶ POST /api/auth/logout/ — blacklists refresh token
- ▶ GET /api/auth/me/ — returns current user profile
- ▶ PUT /api/auth/change-password/ — old password + new password
- ▶ CORS config for Next.js frontend origin
- ▶ Custom permission classes: IsAdminUser, IsMerchantUser, IsSuperAdmin
- ▶ Django admin panel setup for superuser

Claude Code Prompt

Note: Copy the entire block below and paste it into Claude Code. Run from your project root.

You are building the Django 5 + Django REST Framework backend for a Courier Management System.

TASK: Create the full project scaffold and authentication system.

PROJECT STRUCTURE:

```
courier-cms-backend/
├── courier_cms/
│   ├── settings/
│   │   ├── base.py
│   │   ├── development.py
│   │   └── production.py
│   ├── urls.py
│   └── wsgi.py
├── apps/
│   ├── users/           ← Custom user model, auth views
│   ├── parcels/         ← (empty for now, just app init)
│   ├── zones/           ← (empty for now)
│   └── cod/              ← (empty for now)
├── .env.example
├── requirements.txt
└── manage.py
```

REQUIREMENTS (requirements.txt):

```
Django==5.1
djangorestframework==3.15
djangorestframework-simplejwt==5.3
django-cors-headers==4.4
django-enviro==0.11
psycopg2-binary==2.9
Pillow==10.4
```

SETTINGS (base.py):

```
- INSTALLED_APPS includes all apps + rest_framework + corsheaders + simplejwt
- AUTH_USER_MODEL = "users.User"
- REST_FRAMEWORK default auth: JWTAuthentication, default permission: IsAuthenticated
- SIMPLE_JWT: ACCESS_TOKEN_LIFETIME = timedelta(days=1), REFRESH_TOKEN_LIFETIME =
timedelta(days=30), ROTATE_REFRESH_TOKENS = True, BLACKLIST_AFTER_ROTATION = True
- CORS_ALLOWED_ORIGINS from env
```

USER MODEL (apps/users/models.py):

```
class User(AbstractBaseUser, PermissionsMixin):
    email = EmailField(unique=True)
    full_name = CharField(max_length=150)
    phone = CharField(max_length=20, blank=True)
    role = CharField(choices=[("SUPER_ADMIN", "Super
Admin"), ("ADMIN", "Admin"), ("MERCHANT", "Merchant")], max_length=20)
```

```
is_active = BooleanField(default=True)
is_staff = BooleanField(default=False)
date_joined = DateTimeField(auto_now_add=True)
USERNAME_FIELD = "email"
REQUIRED_FIELDS = ["full_name", "role"]

PERMISSION CLASSES (apps/users/permissions.py):
- IsSuperAdmin: role == SUPER_ADMIN
- IsAdminOrSuperAdmin: role in [ADMIN, SUPER_ADMIN]
- IsMerchant: role == MERCHANT

AUTH ENDPOINTS (all under /api/auth/):
POST /login/
  Body: { email, password }
  Returns: { access, refresh, user: { id, email, full_name, role } }

POST /refresh/
  Body: { refresh }
  Returns: { access }

POST /logout/
  Body: { refresh }
  Action: blacklists the refresh token
  Returns: 204

GET /me/
  Returns: { id, email, full_name, phone, role, date_joined }

PUT /change-password/
  Body: { old_password, new_password }
  Returns: 200 or 400 with error

ADMIN USER MANAGEMENT (admin-only):
GET /api/admin/users/          ← list all users (filterable by role)
POST /api/admin/users/        ← create admin user
PATCH /api/admin/users/{id}/ ← update user
DELETE /api/admin/users/{id}/ ← deactivate (set is_active=False)

Write complete, runnable code for all files. Include a management command: python
manage.py seed_admin to create a superuser with email=admin@cms.com password=Admin@1234.

INTEGRATION NOTE: After building, the frontend /login page should call POST
/api/auth/login/ and store the returned { access, refresh, user } in localStorage as
"cms_token" and "cms_user".
```

M5 — Merchant Management API

Backend	Days 19–22	Build merchant profile model, registration endpoints, and admin management APIs. Integrate with frontend merchant pages.
---------	------------	--

Frontend Tasks

- ▶ Replace mock data in /admin/merchants with GET /api/admin/merchants/
- ▶ Approve/Suspend actions call PATCH /api/admin/merchants/{id}/status/
- ▶ Merchant detail page loads from GET /api/admin/merchants/{id}/
- ▶ Merchant /profile page: GET + PUT /api/merchant/profile/
- ▶ Merchant registration: POST /api/auth/register/ (replaces mock signup)

Backend Tasks

- ▶ MerchantProfile model: OneToOne → User, shop_name, pickup_address, district, business_type, status (PENDING/ACTIVE/SUSPENDED), approved_at, approved_by
- ▶ POST /api/auth/register/ — creates User(role=MERCHANT) + MerchantProfile(status=PENDING), sends welcome email via Celery
- ▶ GET /api/merchant/profile/ — returns merchant's own profile
- ▶ PUT /api/merchant/profile/ — update own profile
- ▶ GET /api/admin/merchants/ — list all merchants, filter by status, search by name/phone
- ▶ GET /api/admin/merchants/{id}/ — detail with parcel summary stats + COD balance
- ▶ PATCH /api/admin/merchants/{id}/status/ — body: { status, reason }, updates status, logs action
- ▶ Email task: Celery task sends email on registration + on status change
- ▶ Pagination: 20 per page on list endpoints

Claude Code Prompt

Note: Copy the entire block below and paste it into Claude Code. Run from your project root.

```
You are continuing the Courier Management System Django backend.

TASK: Build the Merchant management module – models, serializers, views, URLs.

MERCHANT PROFILE MODEL (apps/users/models.py – add to existing file):
class MerchantProfile(Model):
    user = OneToOneField(User, on_delete=CASCADE, related_name="merchant_profile")
    shop_name = CharField(max_length=200)
    pickup_address = TextField()
    district = CharField(max_length=100)
    business_type = CharField(max_length=100, blank=True)
    status =
CharField(choices=[ ("PENDING", "Pending"), ("ACTIVE", "Active"), ("SUSPENDED", "Suspended") ],
default="PENDING", max_length=20)
    approved_at = DateTimeField(null=True, blank=True)
    approved_by = ForeignKey(User, null=True, blank=True, on_delete=SET_NULL,
related_name="approved_merchants")
    suspension_reason = TextField(blank=True)
    created_at = DateTimeField(auto_now_add=True)

ENDPOINTS TO BUILD:

PUBLIC:
```

```
POST /api/auth/register/
  Body: { email, full_name, phone, password, shop_name, pickup_address, district,
business_type }
  Action: create User(role=MERCHANT, is_active=True) + MerchantProfile(status=PENDING)
  Returns: 201 { message: "Registration successful. Awaiting admin approval." }
  Side effect: trigger Celery email task to admin

MERCHANT (IsAuthenticated + IsMerchant):
GET  /api/merchant/profile/      → MerchantProfileSerializer (full)
PUT  /api/merchant/profile/      → update own profile (not status, not user email)

ADMIN (IsAdminOrSuperAdmin):
GET  /api/admin/merchants/
  Query params: status, search (name/shop/phone), ordering, page
  Returns: paginated list { id, user.full_name, user.email, user.phone, shop_name,
district, status, created_at }

GET  /api/admin/merchants/{id}/
  Returns: full profile + stats:
    { profile fields, parcel_count, delivered_count, pending_cod_amount }

PATCH /api/admin/merchants/{id}/status/
  Body: { status: "ACTIVE"|"SUSPENDED", reason: "" }
  Guards: only SUPER_ADMIN can approve/reject; ADMIN can suspend
  Action: update status, set approved_at if ACTIVE, set suspension_reason if SUSPENDED
  Returns: updated merchant

SERIALIZERS:
- MerchantListSerializer (for list view — minimal fields)
- MerchantDetailSerializer (for detail view — full fields + stats via
SerializerMethodField)
- MerchantProfileUpdateSerializer (for merchant self-update — excludes status/user fields)
- MerchantRegisterSerializer (for registration — includes User creation in .create())

CELERY TASKS (apps/users/tasks.py):
- send_registration_notification(merchant_id): email to admin
- send_status_update_email(merchant_id, new_status): email to merchant

Include complete URLs, register in courier_cms/urls.py.
Write full runnable code for all files.
```

M6 — Zone & Parcel API

Backend	Days 23–27	Build zones, delivery charges, and full parcel CRUD with status management. Integrate with frontend parcel pages.
---------	------------	---

Frontend Tasks

- ▶ Admin /zones: GET /api/admin/zones/ + POST/PATCH zone
- ▶ Merchant /parcels/new Step 3: GET /api/merchant/zones/ to compute charge preview
- ▶ Admin /parcels: GET /api/admin/parcels/ with filters
- ▶ Admin /parcels/[id]: GET detail + PATCH status + POST remark
- ▶ Merchant /parcels: GET /api/merchant/parcels/ with filters

- ▶ Merchant /parcels/{id}: GET detail + POST cancel
- ▶ Parcel CSV import: POST /api/merchant/parcels/import/

Backend Tasks

- ▶ Zone model: name, districts (ArrayField), regular_charge, express_charge, cod_charge_percent, return_charge
- ▶ Parcel model: tracking_id (auto-generated), merchant (FK), zone (FK), recipient fields, weight, cod_amount, delivery_type, status, created_at
- ▶ ParcelStatusLog model: parcel (FK), from_status, to_status, changed_by (FK User), remark, timestamp
- ▶ GET/POST /api/admin/zones/ + PATCH/DELETE /api/admin/zones/{id}/
- ▶ GET /api/merchant/zones/ — public list for charge calculation
- ▶ GET/POST /api/admin/parcels/ — all parcels across merchants, advanced filters
- ▶ PATCH /api/admin/parcels/{id}/status/ — update status, creates ParcelStatusLog
- ▶ POST /api/admin/parcels/{id}/remarks/ — add internal remark
- ▶ GET/POST /api/merchant/parcels/ — merchant's own parcels only
- ▶ GET /api/merchant/parcels/{id}/ — detail with full status log
- ▶ POST /api/merchant/parcels/{id}/cancel/ — cancel if status is PENDING
- ▶ POST /api/merchant/parcels/import/ — CSV upload, bulk create parcels
- ▶ Tracking ID generation: CMS-YYYY-XXXXXX (random 6 digits, unique check)
- ▶ Charge calculation utility: compute_charge(zone, delivery_type, weight, cod_amount)

Claude Code Prompt

Note: Copy the entire block below and paste it into Claude Code. Run from your project root.

```
You are continuing the Courier Management System Django backend.

TASK: Build the Zones and Parcels modules.

MODELS (create new app files):

apps/zones/models.py:
class Zone(Model):
    name = CharField(max_length=100, unique=True)
    districts = JSONField(default=list) # list of district names
    regular_charge = DecimalField(max_digits=8, decimal_places=2)
    express_charge = DecimalField(max_digits=8, decimal_places=2)
    cod_charge_percent = DecimalField(max_digits=5, decimal_places=2, default=1.00)
    return_charge = DecimalField(max_digits=8, decimal_places=2)
    is_active = BooleanField(default=True)
    created_at = DateTimeField(auto_now_add=True)

apps/parcels/models.py:
PARCEL_STATUS = [ ("PENDING", "Pending"), ("PICKED_UP", "Picked Up"), ("IN_TRANSIT", "In
Transit"),
    ("OUT_FOR_DELIVERY", "Out for
Delivery"), ("DELIVERED", "Delivered"), ("PARTIALLY_DELIVERED", "Partially Delivered"),
    ("RETURN_IN_TRANSIT", "Return in
Transit"), ("RETURNED", "Returned"), ("CANCELLED", "Cancelled")]

class Parcel(Model):
    tracking_id = CharField(max_length=20, unique=True, db_index=True)
    merchant = ForeignKey("users.MerchantProfile", on_delete=PROTECT,
related_name="parcels")
    zone = ForeignKey("zones.Zone", on_delete=PROTECT, related_name="parcels")
    # Recipient
```

```

recipient_name = CharField(max_length=150)
recipient_phone = CharField(max_length=20)
recipient_address = TextField()
recipient_district = CharField(max_length=100)
# Parcel details
delivery_type = CharField(choices=[("REGULAR", "Regular"), ("EXPRESS", "Express")],
max_length=10)
weight = DecimalField(max_digits=6, decimal_places=2)
product_description = TextField(blank=True)
special_instructions = TextField(blank=True)
# Charges (computed at booking time)
cod_amount = DecimalField(max_digits=10, decimal_places=2, default=0)
delivery_charge = DecimalField(max_digits=8, decimal_places=2)
cod_charge = DecimalField(max_digits=8, decimal_places=2, default=0)
total_charge = DecimalField(max_digits=8, decimal_places=2)
# Status
status = CharField(choices=PARCEL_STATUS, default="PENDING", max_length=30)
created_at = DateTimeField(auto_now_add=True)
updated_at = DateTimeField(auto_now=True)

class ParcelStatusLog(Model):
    parcel = ForeignKey(Parcel, on_delete=CASCADE, related_name="status_logs")
    from_status = CharField(max_length=30, blank=True)
    to_status = CharField(max_length=30)
    changed_by = ForeignKey("users.User", on_delete=SET_NULL, null=True)
    remark = TextField(blank=True)
    created_at = DateTimeField(auto_now_add=True)

UTILITY (apps/parcels/utils.py):
def generate_tracking_id():
    # Format: CMS-2025-XXXXXX
    # Ensure uniqueness with retry loop

def compute_charge(zone, delivery_type, weight, cod_amount):
    # base = zone.regular_charge or zone.express_charge
    # weight_extra = max(0, weight - 1) * 10 # 10tk per extra kg
    # cod_charge = max(10, cod_amount * zone.cod_charge_percent / 100)
    # total = base + weight_extra + cod_charge
    # Return dict: { delivery_charge, cod_charge, total_charge }

ENDPOINTS:

ZONES:
GET /api/zones/ ← public, list active zones (for merchant booking)
GET /api/admin/zones/ ← admin: all zones including inactive
POST /api/admin/zones/ ← admin: create zone
PATCH /api/admin/zones/{id}/ ← admin: update zone
DELETE /api/admin/zones/{id}/ ← admin: set is_active=False

PARCELS — Admin:
GET /api/admin/parcels/
    Filters: status, merchant_id, zone_id, delivery_type, date_from, date_to, search (tracking_id/recipient_phone)
    Returns: paginated list with merchant name, recipient, zone, COD, status, date

GET /api/admin/parcels/{id}/ ← full detail + status_logs

PATCH /api/admin/parcels/{id}/status/
    Body: { status, remark }
    Validates allowed status transitions
    Creates ParcelStatusLog entry
    Returns: updated parcel

```

```
POST    /api/admin/parcels/{id}/remarks/
  Body: { remark }
  Creates ParcelStatusLog with same from_status=to_status (remark only, no status change)

PARCELS — Merchant:
GET     /api/merchant/parcels/
  Filters: status, delivery_type, date_from, date_to, search
  Only returns parcels belonging to request.user.merchant_profile

POST    /api/merchant/parcels/
  Body: { recipient_name, recipient_phone, recipient_address, recipient_district,
delivery_type, weight, product_description, special_instructions, cod_amount, zone_id }
  Action: compute charges, generate tracking_id, create Parcel + initial ParcelStatusLog
  Returns: created parcel with tracking_id and charges

GET     /api/merchant/parcels/{id}/          ← detail + status_logs (own parcels only)

POST    /api/merchant/parcels/{id}/cancel/
  Guard: status must be PENDING
  Action: change status to CANCELLED, create log
  Returns: updated parcel

POST    /api/merchant/parcels/import/
  Body: multipart/form-data with file (CSV)
  Parses CSV columns: recipient_name, recipient_phone, recipient_address,
recipient_district, delivery_type, weight, cod_amount, zone_name
  Creates parcels in bulk (use bulk_create with batch_size=50)
  Returns: { created: N, errors: [{row: X, error: "..."}] }
```

Write complete runnable code for all models, serializers, views, URLs.

M7 — COD & Payment API + Reports

Backend	Days 28–32	Build COD disbursement tracking, payment records, and all report endpoints. Integrate with frontend COD and report pages.
---------	------------	---

Frontend Tasks

- Admin /cod: GET /api/admin/cod/summary/ + POST /api/admin/cod/disburse/
- Merchant /cod: GET /api/merchant/cod/summary/ + GET /api/merchant/cod/transactions/
- Admin /reports: GET /api/admin/reports/parcels/ + /merchants/ + /cod/ with CSV export
- Merchant /reports: GET /api/merchant/reports/parcels/ with CSV export
- Export CSV: frontend receives JSON, converts to CSV via existing ExportButton component

Backend Tasks

- CODDisbursement model: merchant (FK), amount, bank_name, reference_number, disbursed_at, disbursed_by (FK User), note
- GET /api/admin/cod/summary/ — merchant list with total_cod, disbursed, pending
- GET /api/admin/cod/{merchant_id}/transactions/ — all disbursements for a merchant
- POST /api/admin/cod/disburse/ — create disbursement record, body: { merchant_id, amount, bank_name, reference_number, note }
- GET /api/merchant/cod/summary/ — own COD summary: total collected, disbursed, pending
- GET /api/merchant/cod/transactions/ — own disbursement history
- GET /api/admin/reports/parcels/?date_from=&date_to=&merchant_id=&zone_id= — aggregated parcel report
- GET /api/admin/reports/merchants/ — merchant performance metrics
- GET /api/admin/reports/zones/ — zone-wise distribution
- GET /api/merchant/reports/parcels/ — own parcel report
- All report endpoints support ?format=csv query param for direct CSV download (using Django StreamingHttpResponse)

Claude Code Prompt

Note: Copy the entire block below and paste it into Claude Code. Run from your project root.

```
You are continuing the Courier Management System Django backend.

TASK: Build the COD/Payment module and all Report endpoints.

COD MODEL (apps/cod/models.py):
class CODDisbursement(Model):
    merchant = ForeignKey("users.MerchantProfile", on_delete=PROTECT,
related_name="cod_disbursements")
    amount = DecimalField(max_digits=12, decimal_places=2)
    bank_name = CharField(max_length=100)
    reference_number = CharField(max_length=100)
    note = TextField(blank=True)
    disbursed_at = DateTimeField()
    disbursed_by = ForeignKey("users.User", on_delete=SET_NULL, null=True,
related_name="cod_disbursements_made")
    created_at = DateTimeField(auto_now_add=True)

COD UTILITY (apps/cod/utils.py):
```

```
def get_merchant_cod_summary(merchant_profile):
    # Total COD = sum of cod_amount for all DELIVERED parcels of this merchant
    # Total Disbursed = sum of all CODDisbursement amounts for this merchant
    # Pending = Total COD - Total Disbursed
    return { total_collected, total_disbursed, pending }
```

COD ENDPOINTS:

Admin:

GET /api/admin/cod/summary/
Returns paginated list of merchants with COD summary:
[{ merchant_id, merchant_name, shop_name, total_collected, total_disbursed, pending, last_disbursement_date }]
Query: search (merchant name), has_pending (bool filter)

GET /api/admin/cod/{merchant_id}/transactions/
Returns: merchant info + list of disbursements (paginated)

POST /api/admin/cod/disburse/
Body: { merchant_id, amount, bank_name, reference_number, disbursed_at, note }
Validates: amount <= pending COD of merchant
Creates CODDisbursement
Returns: created disbursement

Merchant:

GET /api/merchant/cod/summary/
Returns: { total_collected, total_disbursed, pending } for own parcels

GET /api/merchant/cod/transactions/
Returns: paginated list of own CODDisbursement records

REPORT ENDPOINTS:

All reports support ?format=csv for CSV download. When format=csv:

- Return StreamingHttpResponse with Content-Type text/csv
- Content-Disposition: attachment; filename="report_YYYYMMDD.csv"

GET /api/admin/reports/parcels/
Params: date_from, date_to, merchant_id, zone_id, status, delivery_type
Returns (JSON): {
 summary: { total, delivered, returned, cancelled, pending, success_rate, total_cod_collected },
 by_status: [{ status, count, percentage }],
 by_zone: [{ zone_name, count }],
 by_day: [{ date, count }],
 parcels: [paginated list]
}

GET /api/admin/reports/merchants/
Params: date_from, date_to
Returns: [{ merchant_name, shop_name, total_parcels, delivered, returned, success_rate, total_cod }]

GET /api/admin/reports/zones/
Params: date_from, date_to
Returns: [{ zone_name, total_parcels, delivered, returned, total_revenue }]

GET /api/merchant/reports/parcels/
Params: date_from, date_to, status, delivery_type
Returns: same structure as admin parcel report but filtered to own parcels

Write complete code for all models, serializers, views, URLs. Use Django ORM aggregation (Count, Sum, annotate) for all report queries. No raw SQL.

M8 — Email Notifications + PDF Invoice

Backend	Days 33–37	Build Celery-based email notification system and PDF parcel invoice generation. Integrate download button on frontend.
---------	------------	--

Frontend Tasks

- ▶ Merchant /parcels/[id]: Download Invoice button calls GET /api/parcels/{id}/invoice/ and triggers browser download
- ▶ Email notifications fire automatically on backend events (no frontend change needed)

Backend Tasks

- ▶ Celery + Redis setup in Django settings
- ▶ Email templates (HTML): registration_pending.html, merchant_approved.html, merchant_suspended.html, parcel_booked.html, parcel_status_changed.html
- ▶ Celery tasks: send_merchant_registration_email, send_merchant_status_change_email, send_parcel_booked_email, send_parcel_status_change_email
- ▶ Hook tasks into: auth/register view, merchant status update view, parcel creation view, parcel status update view
- ▶ GET /api/parcels/{tracking_id}/invoice/ — generates PDF using ReportLab, returns as application/pdf response
- ▶ PDF content: company logo placeholder, tracking ID + QR code (qrcode library), recipient info, parcel details, charges breakdown, status
- ▶ GET /api/admin/parcels/manifest/ — bulk PDF manifest for multiple parcel IDs

Claude Code Prompt

Note: Copy the entire block below and paste it into Claude Code. Run from your project root.

```
You are continuing the Courier Management System Django backend.

TASK: Set up Celery for background tasks, build email notification system, and add PDF invoice generation.

CELERY SETUP:
1. Install: celery==5.4, redis==5.0, django-celery-results==2.5
2. Create courier_cms/celery.py with standard Django Celery config
3. Update courier_cms/__init__.py to auto-discover tasks
4. Add to settings: CELERY_BROKER_URL = env("REDIS_URL", default="redis://localhost:6379/0")

EMAIL TEMPLATES (apps/users/templates/emails/):
Create HTML email templates with inline CSS. Simple, clean design. Use {{ variable }} placeholders.

1. registration_pending.html
   Subject: "New Merchant Registration — {shop_name}"
```

```

    To: admin email
    Content: merchant name, shop name, email, phone, registration date, link to admin panel

2. merchant_approved.html
    Subject: "Your Account Has Been Approved - {shop_name}"
    To: merchant email
    Content: welcome message, login link

3. merchant_suspended.html
    Subject: "Account Notice - {shop_name}"
    To: merchant email
    Content: suspension reason, contact info

4. parcel_booked.html
    Subject: "Parcel Booked - {tracking_id}"
    To: merchant email
    Content: tracking ID, recipient name, delivery type, charges, expected delivery note

5. parcel_status_changed.html
    Subject: "Parcel Update - {tracking_id} is now {status}"
    To: merchant email
    Content: parcel summary, new status, what it means, tracking link

CELERY TASKS (apps/users/tasks.py and apps/parcels/tasks.py):
Each task:
- Receives an ID, queries DB fresh (don't pass model objects to Celery)
- Uses Django's send_mail or EmailMultiAlternatives for HTML email
- Handles exceptions gracefully (log errors, don't crash)
- Add retry: max_retries=3, countdown=60

Hook tasks:
- After register → send_merchant_registration_email.delay(merchant_id)
- After merchant status change → send_merchant_status_change_email.delay(merchant_id)
- After parcel create → send_parcel_booked_email.delay(parcel_id)
- After parcel status update → send_parcel_status_change_email.delay(parcel_id)

PDF INVOICE (apps/parcels/views.py - new view):
Install: reportlab==4.2, qrcode==7.4

GET /api/parcels/{tracking_id}/invoice/
- Permission: IsAuthenticated (merchant can only get own parcels, admin can get any)
- Generates PDF using ReportLab with:
    Header: "COURIER MANAGEMENT SYSTEM" + tracking_id
    QR Code: encode tracking_id as QR image, embed in PDF
    Section: Sender (merchant shop name, pickup address)
    Section: Recipient (name, phone, address, district)
    Section: Parcel Details (weight, delivery type, zone, description)
    Section: Charges (delivery charge, COD charge, total charge, COD amount)
    Section: Current Status + date
    Footer: "Thank you for using our service"
- Returns: HttpResponse(pdf_buffer, content_type="application/pdf") with
Content-Disposition: attachment; filename="{tracking_id}_invoice.pdf"

GET /api/admin/parcels/manifest/
- Query param: ids=CMS-2025-001,CMS-2025-002,...
- Generates a multi-page PDF with one parcel label per page
- Each page: large tracking ID text, QR code, recipient info, delivery type badge
- Returns as PDF download

Write complete, runnable code for all tasks, views, and PDF generation.

```


M9 — QA, Testing & Production Deployment

Testing & Deploy	Days 38–45	Write tests, fix bugs, deploy both backend and frontend to VPS with Nginx, SSL, and systemd services.
------------------	------------	---

Frontend Tasks

- ▶ End-to-end flow testing: register merchant → admin approves → merchant books parcel → admin updates status → merchant views tracking
- ▶ Fix any broken API integrations found during testing
- ▶ Add loading states and error boundaries on all pages
- ▶ Environment config: .env.local for dev, .env.production for VPS
- ▶ Build: next build — fix any TypeScript or build errors

Backend Tasks

- ▶ Write pytest test cases: auth (login/logout/refresh), merchant CRUD, parcel lifecycle, COD calculation, zone charges
- ▶ Fix any bugs found in integration testing
- ▶ Add django-silk or django-debug-toolbar for performance profiling
- ▶ Add database indexes on frequently queried fields
- ▶ Production settings: DEBUG=False, ALLOWED_HOSTS, STATIC_FILES via WhiteNoise

Claude Code Prompt

Note: Copy the entire block below and paste it into Claude Code. Run from your project root.

```
You are finalizing the Courier Management System for production deployment.

TASK 1: Write comprehensive tests for the Django backend.

TEST FILES TO CREATE:

tests/test_auth.py:
- test_login_success, test_login_wrong_password, test_login_inactive_user
- test_refresh_token, test_logout_blacklists_token
- test_change_password_success, test_change_password_wrong_old

tests/test_merchants.py:
- test_merchant_registration, test_registration_duplicate_email
- test_admin_approve_merchant, test_admin_suspend_merchant
- test_merchant_cannot_access_admin_endpoints
- test_merchant_update_own_profile

tests/test_parcels.py:
- test_create_parcel_charge_calculation
- test_tracking_id_unique_format
- test_parcel_status_transition_valid, test_parcel_status_invalid_transition
- test_merchant_can_cancel_pending, test_cannot_cancel_delivered
- test_csv_import_success, test_csv_import_with_errors
- test_merchant_cannot_see_other_merchant_parcels

tests/test_cod.py:
- test_cod_summary_calculation
- test_disbursement_cannot_exceed_pending
- test_report_aggregations
```

Use `pytest-django`, `factory_boy` for test data. Each test should be fully isolated.

TASK 2: Production deployment guide. Generate these shell scripts:

```
deploy_backend.sh:
#!/bin/bash
# 1. Git pull latest
# 2. Activate virtualenv
# 3. pip install -r requirements.txt
# 4. python manage.py migrate
# 5. python manage.py collectstatic --noinput
# 6. sudo systemctl restart courier-cms-backend
# 7. sudo systemctl restart courier-cms-celery
echo "Backend deployed"

/etc/systemd/system/courier-cms-backend.service:
[Unit]
Description=Courier CMS Django Backend
After=network.target

[Service]
User=www-data
WorkingDirectory=/var/www/courier-cms-backend
ExecStart=/var/www/courier-cms-backend/venv/bin/gunicorn courier_cms.wsgi:application
--bind 127.0.0.1:8001 --workers 3
EnvironmentFile=/var/www/courier-cms-backend/.env
Restart=always

[Install]
WantedBy=multi-user.target

/etc/systemd/system/courier-cms-celery.service:
[Unit]
Description=Courier CMS Celery Worker
After=network.target

[Service]
User=www-data
WorkingDirectory=/var/www/courier-cms-backend
ExecStart=/var/www/courier-cms-backend/venv/bin/celery -A courier_cms worker
--loglevel=info
EnvironmentFile=/var/www/courier-cms-backend/.env
Restart=always

[Install]
WantedBy=multi-user.target

/etc/nginx/sites-available/courier-cms:
server {
    listen 80;
    server_name yourdomain.com;
    # Redirect to HTTPS
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl;
    server_name yourdomain.com;

    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;

    # Frontend (Next.js on port 3000)
```

```
location / {
    proxy_pass http://127.0.0.1:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}

# Backend API
location /api/ {
    proxy_pass http://127.0.0.1:8001;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}

# Django static/media
location /static/ { alias /var/www/courier-cms-backend/staticfiles/; }
location /media/ { alias /var/www/courier-cms-backend/media/; }
}

Frontend deployment (.env.production):
NEXT_PUBLIC_API_URL=https://yourdomain.com/api

deploy_frontend.sh:
#!/bin/bash
npm install
npm run build
sudo systemctl restart courier-cms-frontend

/etc/systemd/system/courier-cms-frontend.service:
[Unit]
Description=Courier CMS Next.js Frontend

[Service]
User=www-data
WorkingDirectory=/var/www/courier-cms-frontend
ExecStart=/usr/bin/node server.js
Environment=PORT=3000
Environment=NODE_ENV=production
Restart=always

[Install]
WantedBy=multi-user.target

Also generate: DEPLOYMENT_CHECKLIST.md with step-by-step checklist for going live.
```

3. API Endpoint Reference

Complete list of all API endpoints grouped by module. Use this as a quick reference during frontend integration.

Method	Endpoint	Access	Description
POST	/api/auth/login/	Public	Login — returns JWT tokens + user info
POST	/api/auth/register/	Public	Merchant registration
POST	/api/auth/refresh/	Public	Refresh access token
POST	/api/auth/logout/	Auth	Logout — blacklist refresh token
GET	/api/auth/me/	Auth	Get current user profile
PUT	/api/auth/change-password/	Auth	Change password
GET	/api/admin/users/	Admin	List all admin users
POST	/api/admin/users/	SuperAdmin	Create admin user
PATCH	/api/admin/users/{id}/	SuperAdmin	Update admin user
GET	/api/admin/merchants/	Admin	List all merchants (filterable)
GET	/api/admin/merchants/{id}/	Admin	Merchant detail + stats
PATCH	/api/admin/merchants/{id}/status/	Admin	Approve / suspend merchant
GET	/api/merchant/profile/	Merchant	Get own merchant profile
PUT	/api/merchant/profile/	Merchant	Update own profile
GET	/api/zones/	Public	List active zones (for booking)
GET	/api/admin/zones/	Admin	Admin zone list
POST	/api/admin/zones/	Admin	Create zone
PATCH	/api/admin/zones/{id}/	Admin	Update zone
GET	/api/admin/parcels/	Admin	All parcels (advanced filters)
GET	/api/admin/parcels/{id}/	Admin	Parcel detail + status log
PATCH	/api/admin/parcels/{id}/status/	Admin	Update parcel status
POST	/api/admin/parcels/{id}/remarks/	Admin	Add internal remark
GET	/api/admin/parcels/manifest/	Admin	Bulk PDF label download
GET	/api/merchant/parcels/	Merchant	Own parcels (filterable)
POST	/api/merchant/parcels/	Merchant	Book new parcel
GET	/api/merchant/parcels/{id}/	Merchant	Own parcel detail
POST	/api/merchant/parcels/{id}/cancel/	Merchant	Cancel pending parcel
POST	/api/merchant/parcels/import/	Merchant	Bulk CSV import
GET	/api/parcels/{tracking_id}/invoice/	Auth	Download parcel PDF invoice
GET	/api/admin/cod/summary/	Admin	All merchants COD balances

GET	/api/admin/cod/{merchant_id}/transactions/	Admin	Merchant disbursement history
POST	/api/admin/cod/disburse/	Admin	Record COD disbursement
GET	/api/merchant/cod/summary/	Merchant	Own COD summary
GET	/api/merchant/cod/transactions/	Merchant	Own disbursement history
GET	/api/admin/reports/parcels/	Admin	Parcel report (supports ?format=csv)
GET	/api/admin/reports/merchants/	Admin	Merchant performance report
GET	/api/admin/reports/zones/	Admin	Zone distribution report
GET	/api/merchant/reports/parcels/	Merchant	Own parcel report

4. Database Model Reference

Quick reference for all database models and their key fields. Use these when reviewing generated code or writing manual queries.

User (apps/users/)

- id (UUID)
- email (unique)
- full_name
- phone
- role (SUPER_ADMIN/ADMIN/MERCHANT)
- is_active
- is_staff
- date_joined

MerchantProfile (apps/users/)

- id
- user (OneToOne → User)
- shop_name
- pickup_address
- district
- business_type
- status (PENDING/ACTIVE/SUSPENDED)
- approved_at
- approved_by (FK User)
- suspension_reason
- created_at

Zone (apps/zones/)

- id
- name (unique)
- districts (JSONField list)
- regular_charge
- express_charge
- cod_charge_percent
- return_charge
- is_active
- created_at

Parcel (apps/parcels/)

- id
- tracking_id (unique, indexed)
- merchant (FK MerchantProfile)
- zone (FK Zone)
- recipient_name
- recipient_phone
- recipient_address

- recipient_district
- delivery_type (REGULAR/EXPRESS)
- weight
- product_description
- special_instructions
- cod_amount
- delivery_charge
- cod_charge
- total_charge
- status
- created_at
- updated_at

ParcelStatusLog (apps/parcels/)

- id
- parcel (FK Parcel)
- from_status
- to_status
- changed_by (FK User)
- remark
- created_at

CODDisbursement (apps/cod/)

- id
- merchant (FK MerchantProfile)
- amount
- bank_name
- reference_number
- note
- disbursed_at
- disbursed_by (FK User)
- created_at

5. Completion Checklist

Use this checklist to track progress. Check off each item before moving to the next module.

Module	Checklist Item
M1	☐ Next.js project scaffold, folder structure, shared components, mock auth working
M1	☐ Admin and Merchant sidebars functional, route guards working, Axios instance set up
M2	☐ All Admin pages built with mock data: dashboard, merchants, parcels, zones, COD, reports
M2	☐ Charts rendering correctly (recharts), StatusBadge component, modals functional
M3	☐ All Merchant pages built: booking form (3 steps), tracking, COD view, CSV import UI
M3	☐ Parcel booking charge calculation preview working in Step 3
M4	☐ Django project running, PostgreSQL connected, migrations applied
M4	☐ Auth endpoints tested: login, refresh, logout, me, change-password
M4	☐ Frontend login replaced with real API call — JWT stored in localStorage
M5	☐ MerchantProfile model migrated, registration endpoint working
M5	☐ Admin merchant list + approve/suspend working end-to-end
M5	☐ Merchant self-registration flow tested: register → admin approves → merchant logs in
M6	☐ Zone model migrated, zone CRUD working for admin
M6	☐ Parcel creation working: tracking ID generated, charges computed correctly
M6	☐ Full parcel lifecycle tested: create → status updates → cancel
M6	☐ CSV import endpoint working, errors returned per row
M7	☐ COD summary calculations correct (delivered parcels only)
M7	☐ Disbursement endpoint validates amount does not exceed pending
M7	☐ Report endpoints return correct aggregated data
M7	☐ CSV export working from report endpoints
M8	☐ Celery worker running, emails sending on all trigger events
M8	☐ PDF invoice generating with QR code, downloadable from frontend
M9	☐ All pytest tests passing (auth, merchants, parcels, COD)
M9	☐ Next.js production build succeeds (npm run build)
M9	☐ Nginx config applied, SSL certificate obtained
M9	☐ Both backend and frontend systemd services running and auto-restarting
M9	☐ Full end-to-end flow tested on production domain